Profiling & Parallelization

Lecture 19

Dr. Colin Rundel

Profiling

Measure first

Intuition about where a program spends its time is often wrong.

Before rewriting anything,

- profile the whole task to find the slow part,
- benchmark alternatives for that part,
- then decide whether parallelization is worth it.

Most of the time the fix is a better algorithm or a vectorized call, not more cores.

A first attempt

The sampling distribution of a regression slope. `sim_slope()` simulates one dataset and hands it to a function that estimates the slope, and we repeat that 20,000 times,

```r
sim_slope = function(n, slope, beta = 0.5) {
  x = rnorm(n)
  y = beta * x + rnorm(n)
  slope(x, y)
}
slope_lm = function(x, y) lm(y ~ x) |> coef() |> _[["x"]]
```

```r
system.time({ slopes = map_dbl(seq_len(2e4), \(i) sim_slope(100, slope_lm)) })
```

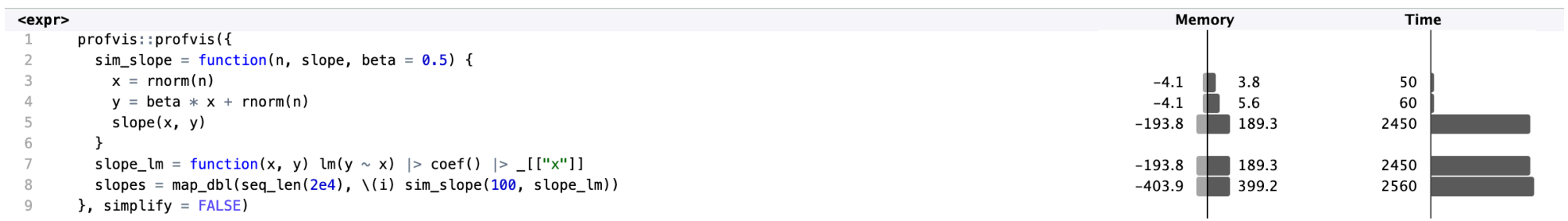
```
   user  system elapsed
  2.465   0.015   2.482
```

A few seconds for 20,000 fits of 100 points each. Is it the random numbers, the fit, or the map?

profvis

```r
profvis::profvis({
  sim_slope = function(n, slope, beta = 0.5) {
    x = rnorm(n)
    y = beta * x + rnorm(n)
    slope(x, y)
  }
  slope_lm = function(x, y) lm(y ~ x) |> coef() |> _[["x"]]
  slopes = map_dbl(seq_len(2e4), \(i) sim_slope(100, slope_lm))
}, simplify = FALSE)
```

Line profile | Flame graph



Reading the profile

`profvis::profvis()` runs an expression while recording the call stack every 10 ms, then reports how often each line and function was on the stack. The functions are defined inside the profiled block so their lines appear in the code panel.

- The code panel attributes time and memory to each line. Nearly all of it lands on the `lm()` line, and the random draws barely register.
- The flame graph shows the call stack over time, zoomed here to a 175 ms window. Inside `lm()` the time goes to `model.frame()`, `model.matrix()`, and evaluating the formula. `lm.fit()`, which does the arithmetic, is a thin slice.
- Hovering a bar gives the function and its time, and the Data tab gives the same information as a tree.

`profvis` is a sampling profiler built on `Rprof()`. Code that runs for less than a few samples will not show up, so profile a

The fix

The formula interface costs about 100 µs per call however small the data. `lm.fit()` takes the design matrix directly, and a simple regression slope is just `cov(x, y) / var(x)`,

```r
slope_fit = function(x, y) lm.fit(cbind(1, x), y) |> _$coefficients[[2]]
slope_cov = function(x, y) cov(x, y) / var(x)
```

```r
system.time({ slopes = map_dbl(seq_len(2e4), \(i) sim_slope(100, slope_fit)) })
```

```
   user  system elapsed
  0.329   0.014   0.346
```

```r
system.time({ slopes = map_dbl(seq_len(2e4), \(i) sim_slope(100, slope_cov)) })
```

```
   user  system elapsed
  0.280   0.001   0.282
```

Both are about ten times faster. The profile pointed at the overhead around the fit, not the fit itself.

Benchmarking

bench::mark()

Once there are alternatives, `bench::mark()` runs each expression repeatedly and summarizes the timings and memory use. By default it first checks that every expression returns the same result,

```r
x = rnorm(100)
y = 0.5 * x + rnorm(100)
```

```r
(b = bench::mark(
  lm = slope_lm(x, y),
  lm.fit = slope_fit(x, y),
  cov_var = slope_cov(x, y)
))
```

```
# A tibble: 3 × 6
  expression      min   median `itr/sec` mem_alloc `gc/sec`
  <bch:expr> <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
1 lm         106.93µs 118.65µs     7930.  437.27KB     14.4
2 lm.fit       8.86µs     10µs    88880.    7.34KB     17.8
3 cov_var       7.5µs   8.36µs   110221.        0B     11.0
```

`check = TRUE` compares results with `all.equal()`. Pass a function instead when differences in order or class are

What is reported

Column	Meaning
`min`, `median`	Fastest and typical time per iteration
`itr/sec`	Iterations per second, the inverse of the mean time
`mem_alloc`	Memory allocated by one iteration
`gc/sec`	Garbage collections per second
`n_itr`	Iterations retained after garbage collection filtering
`n_gc`	Garbage collections across all iterations
`total_time`	Total time of the retained iterations

Each expression runs until `min_time` (0.5 s) has passed or `max_iterations` (10,000) is reached, so slow expressions get few iterations and fast ones get many.

`summary(b, relative = TRUE)` divides each numeric summary column by its minimum. For `itr/sec`, the slowest

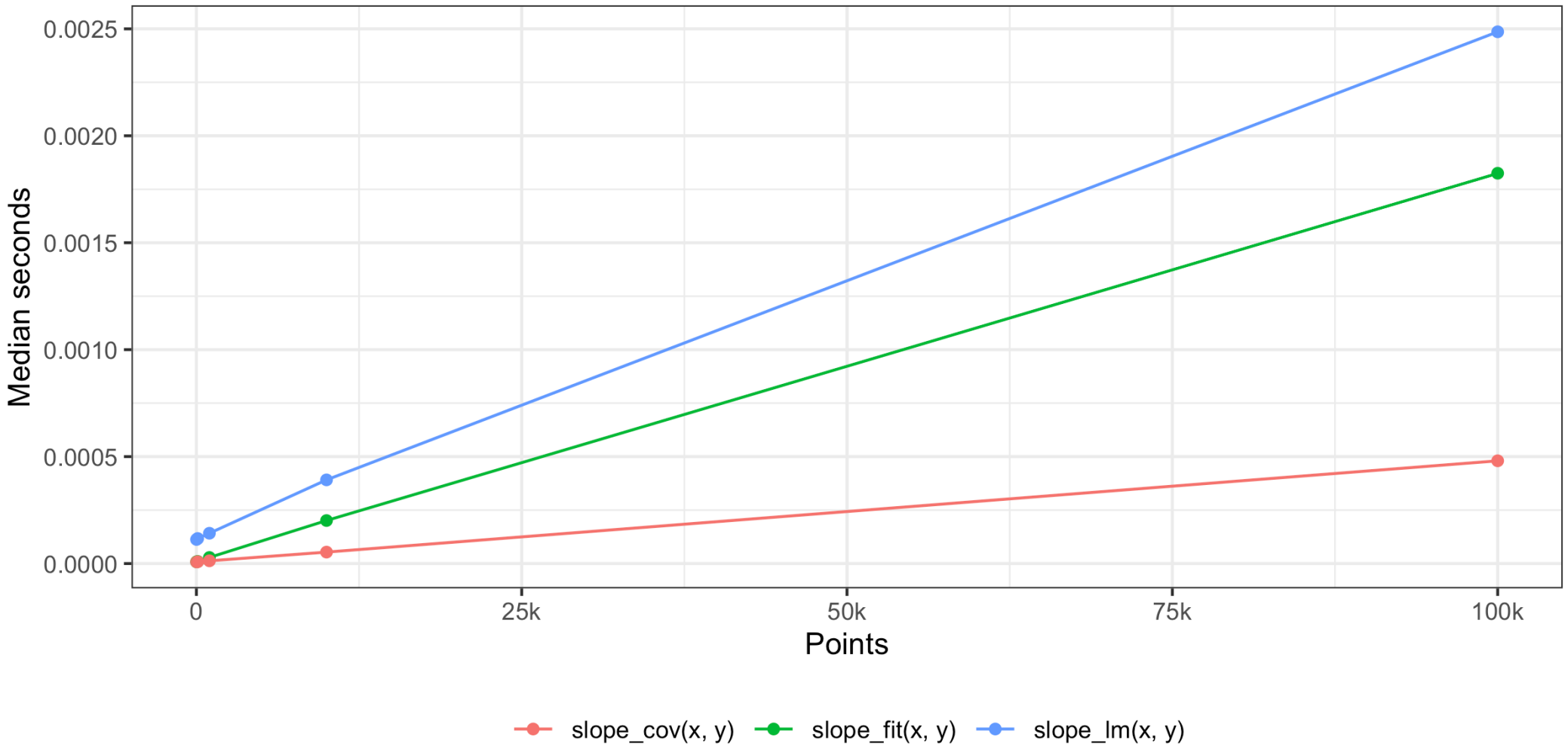
`bench::press()`

How does the cost of one fit change with the number of points?

```r
p = bench::press(
  n = c(10, 100, 1000, 10000, 100000),
  {
    x = rnorm(n)
    y = 0.5 * x + rnorm(n)
    bench::mark(
      lm = slope_lm(x, y),
      lm.fit = slope_fit(x, y),
      cov_var = slope_cov(x, y),
      min_time = 0.1
    )
  }
)
```

`press()` runs the benchmark at every combination of parameter values, here just the five sizes.

Scaling the same task



Up to a few thousand points the fixed cost of `lm()` dominates. Past that the arithmetic grows with `n` and the three converge.

Parallelization

Why

Most machines have many cores and R uses one of them (mostly). Tasks that split into independent pieces can make use of the multiple cores,

- simulation studies and bootstrap resampling,
- cross validation and tuning grids,
- fitting the same model to many groups,
- processing many files.

Each piece has a cost beyond the computation itself, starting a process, sending it the data it needs, and sending the result back. Parallelization helps when the pieces are slow relative to that overhead.

`detectCores()`

The `parallel` package is part of base R and reports the number of cores it can see, as does `os` in Python,

```r
parallel::detectCores()
```

```
[1] 14
```

```python
import os
os.cpu_count(), os.process_cpu_count()
```

```
(14, 14)
```

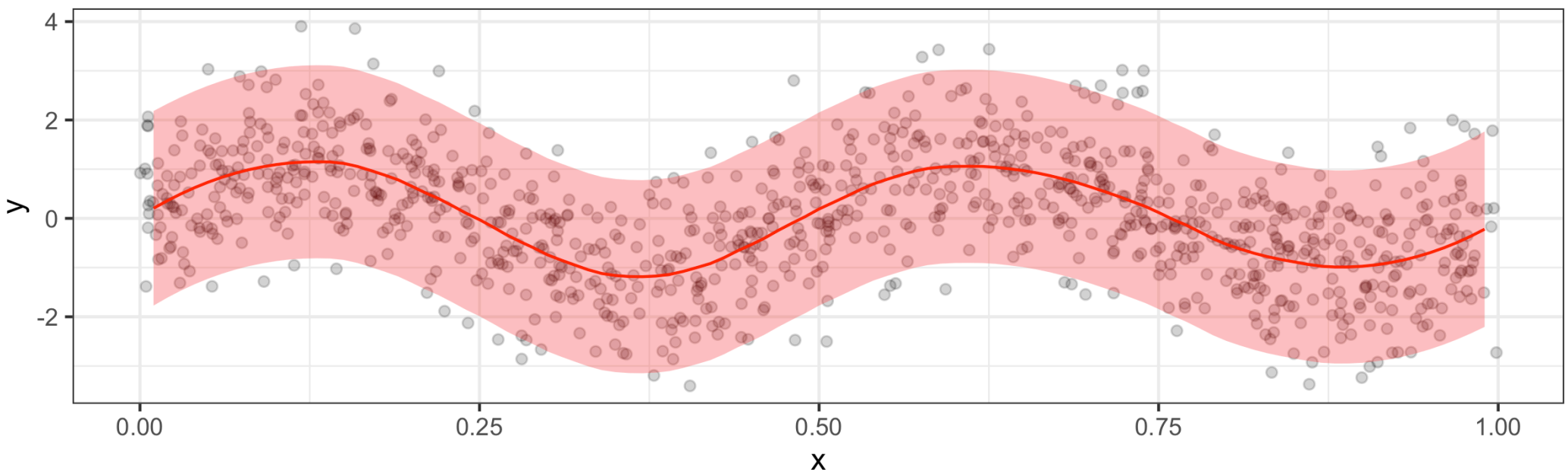
Treat this number as an upper bound rather than a target,

- it counts logical cores. `detectCores(logical = FALSE)` gives the physical count,
- recent CPUs mix performance and efficiency cores, and work on the slow cores runs slower,
- a multithreaded BLAS, `data.table`, `duckdb`, and `polars` already use several cores, so workers running them oversubscribe the machine,
- on a shared server the count is for the whole machine, and other users need cores too.

A realistic task

Bootstrapping refits a model to repeated resamples of the data. Here the model is a `loess` smooth of 1,000 noisy points on two cycles of a sine curve, and we want a bootstrap prediction band for new observations on a grid of `x` values,

```r
set.seed(3212016)
d = tibble(x = runif(1000)) |>
  mutate(y = sin(4 * pi * x) + rnorm(1000))
grid = seq(0.01, 0.99, by = 0.01)
```



One replicate

`boot_fit()` fits the smooth to one resample, predicts on the grid, and adds a resampled residual to each prediction to simulate a new observation. The resamples themselves are drawn up front,

```r
boot_fit = function(b, grid, span = 0.3) {
  fit = loess(y ~ x, data = b, span = span)
  pred = predict(fit, newdata = tibble::tibble(x = grid))
  tibble::tibble(
    span = span,
    x = grid,
    y_new = pred + sample(residuals(fit), length(grid), replace = TRUE)
  )
}
boots = map(
  seq_len(1000),
  \(i) slice_sample(d, prop = 1, replace = TRUE)
)
```

```r
boot_fit(boots[[1]], grid)
```

```
# A tibble: 99 × 3
   span     x  y_new
  <dbl> <dbl>  <dbl>
1   0.3  0.01  0.890
2   0.3  0.02 -0.599
3   0.3  0.03 -0.687
4   0.3  0.04  1.36
5   0.3  0.05  1.15
6   0.3  0.06 -0.783
# ℹ 93 more rows
```

The `tibble` calls in `boot_fit()` are namespaced because it will later run in fresh R processes where the package is not

mclapply()

Each replicate is independent and 1,000 take a while,

```R
(time_seq = system.time(lapply(boots, boot_fit, grid = grid)))
```

```
   user  system elapsed 
  2.782   0.033   2.828 
```

`parallel::mclapply()` is `lapply()` across forked copies of the session,

```R
(time_mc2 = system.time(
  mclapply(boots, boot_fit, grid = grid,
           mc.cores = 2)
))
```

```
   user  system elapsed 
  0.003   0.022   1.455 
```

```R
(time_mc4 = system.time(
  mclapply(boots, boot_fit, grid = grid,
           mc.cores = 4)
))
```

```
   user  system elapsed 
  1.506   0.136   0.841 
```

```R
(time_mc6 = system.time(
  mclapply(boots, boot_fit, grid = grid,
           mc.cores = 6)
))
```

```
   user  system elapsed 
  1.064   0.152   0.625 
```

```R
(time_mc8 = system.time(
  mclapply(boots, boot_fit, grid = grid,
           mc.cores = 8)
))
```

```
   user  system elapsed 
  2.377   0.342   0.485 
```

Forking

`fork()` duplicates the running process. The child is a copy of the parent at that instant, with the same objects, packages, and point of execution.

- The copy is cheap. Memory is shared copy-on-write, so data that is only read costs nothing extra per worker.
- Nothing is sent to a worker. Each child computes its share, sends the result back through a pipe, and exits.
- Windows has no `fork()`, so `mclapply()` runs sequentially there. `parLapply()` and `mirai` start fresh R sessions instead and send them what they need.
- Only the calling thread is copied, so forking from a session with other threads (a GUI, a multithreaded BLAS) can leave the child broken. RStudio warns about this.

Forking & Random numbers

Each forked worker starts from a copy of the parent's RNG state. With the default generator, `mclapply()` reseeds each worker from the clock, so results change from run to run even after `set.seed()`.

```r
draw = function() unlist(mclapply(1:3, \(i) rnorm(1), mc.cores = 2))
```

```r
set.seed(1234)
draw()
```

```
[1]  1.3289478  0.7666974 -0.2195417
```

```r
set.seed(1234)
draw()
```

```
[1] -0.4660414 -0.4698863 -2.1040503
```

```r
RNGkind("L'Ecuyer-CMRG")
set.seed(1234)
draw()
```

```
[1] -0.2208308  0.3109666  1.1507972
```

```r
set.seed(1234)
draw()
```

```
[1] -0.2208308  0.3109666  1.1507972
```

L'Ecuyer-CMRG is a generator that can produce independent streams, which is what the workers need. `mclapply()` gives


mirai
ミライ
ミライ
ミライ
みらい
未来

`mirai`

`mirai` is a framework for asynchronous and parallel evaluation of R code,

- code is sent to persistent background R processes (daemons) and the result is collected when needed,
- works the same on every operating system, and daemons can run on remote machines or an HPC cluster,
- errors in a task are returned as values rather than stopping everything,
- it is the engine behind parallel maps in `purrr` and `tidymodels`, and asynchronous tasks in Shiny.

daemons()

Daemons are set once per session. They sit idle until given work, so there is no reason to start and stop them.

```r
daemons(4)
info()
```

```
connections  cumulative    awaiting   executing   completed
          4           4           0           0           0
```

- `daemons(0)` stops and removes all existing daemons.
- Calling `daemons(n)` creates `n` new daemons.
- `daemons(sync = TRUE)` runs every task in the current session, which is useful for debugging.

mirai()

`mirai()` sends an expression to a daemon and returns immediately. The session is free while the task runs.

A mirai is unresolved until the daemon returns a value. `m[]` waits (blocks) for the result, and `m$data` holds it once resolved.

```r
(m = mirai(
  boot_fit(b, grid),
  boot_fit = boot_fit,
  b = boots[[1]], grid = grid
))
```

```
< mirai [] >
```

```r
unresolved(m)
```

```
[1] TRUE
```

```r
m[]
```

```
# A tibble: 99 × 3
   span     x   y_new
  <dbl> <dbl>   <dbl>
1   0.3  0.01  0.536
2   0.3  0.02  1.40
3   0.3  0.03 -0.0720
4   0.3  0.04 -0.0385
5   0.3  0.05  1.28
6   0.3  0.06  0.667
# ℹ 93 more rows
```

```r
m
```

```
< mirai [$data] >
```

What a daemon can see

The expression runs in a fresh R process, not in your session, so everything it uses must be sent along or loaded there.

```R
y = 10
```

```R
mirai(x + y, x = 1)[]
```

```
'miraiError' chr Error: object 'y' not found
```

```R
mirai(x + y, x = 1, y = y)[]
```

```
[1] 11
```

Named arguments after the expression are assigned in the daemon. Use `.args = list(...)` instead for objects that should be local to that one evaluation.

This is why the previous call passed `boot_fit`, `b`, and `grid`. A forked worker would have inherited them, but a daemon starts empty, so the function and its data must be shipped along with the expression.

Only pass what the task needs. Every object is serialized and sent to the daemon, so a large data frame that is not used

Packages on daemons

Functions from packages must be namespaced, sent along like any other object, or the package attached on the daemons with `everywhere()`,

```r
tb = tibble(a = 1:3)
```

```r
mirai(mutate(tb, b = a * 2), tb = tb)[]
```

```
'miraiError' chr Error in mutate(tb, b = a *
2): could not find function "mutate"
```

```r
mirai(dplyr::mutate(tb, b = a * 2),
      tb = tb)[]
```

```
# A tibble: 3 × 2
      a     b
  <int> <dbl>
1     1     2
2     2     4
3     3     6
```

```r
mirai(mutate(tb, b = a * 2),
      tb = tb, mutate = dplyr::mutate)[]
```

```
# A tibble: 3 × 2
      a     b
  <int> <dbl>
1     1     2
2     2     4
3     3     6
```

```r
everywhere(library(dplyr))
mirai(mutate(tb, b = a * 2), tb = tb)[]
```

```
# A tibble: 3 × 2
      a     b
  <int> <dbl>
1     1     2
2     2     4
3     3     6
```

Between tasks

Each task runs in a fresh environment, and afterwards the daemon restores its global environment, packages, and options. With one daemon both tasks below run in the same process,

```r
daemons(1)
```

```r
mirai({
  library(stringr)
  leftover = 42
})[]
```

```
[1] 42
```

```r
mirai(c(
  stringr = "package:stringr" %in% search(),
  leftover = exists("leftover")
))[]
```

```
stringr leftover
  FALSE    FALSE
```

mirai_map()

`mirai_map()` is `lapply()` where each element becomes its own mirai. Arguments that are the same for every task go in `.args`,

```r
(mm = mirai_map(boots, boot_fit, .args = list(grid = grid)))
```

```
< mirai map [0/1000] >
```

```r
res = mm[]
bind_rows(res, .id = "rep")
```

```
# A tibble: 99,000 × 4
  rep    span     x  y_new
  <chr> <dbl> <dbl>  <dbl>
1 1       0.3  0.01 -1.81
2 1       0.3  0.02  0.126
3 1       0.3  0.03 -0.462
4 1       0.3  0.04  0.111
5 1       0.3  0.05  1.10
6 1       0.3  0.06  1.97
# ℹ 98,994 more rows
```

Time the whole map

`mclapply()` blocks until every result is back, so wrapping it in `system.time()` measured the whole job. `mirai_map()` returns as soon as the tasks are submitted, so the timer has to run from submission through collection with `[]`.

Starting daemons also takes a moment, since each one is a new R process. `everywhere(NULL, .min = workers)` evaluates a no-op on every daemon, and collecting it waits until all of them have connected, which keeps startup out of the timing.

```R
time_mirai_map = function(workers) {
  daemons(workers)
  everywhere(NULL, .min = workers)[]
  system.time(
    mirai_map(boots, boot_fit, .args = list(grid = grid))[]
  )
}
time_mirai = map(c(2, 4, 6, 8), time_mirai_map)
map_dbl(time_mirai, "elapsed")
```

```
[1] 1.838 1.011 0.811 0.615
```

Compare elapsed times

```
# A tibble: 9 × 4
  method    workers seconds speedup
  <chr>       <dbl>   <dbl>   <dbl>
1 lapply          1   2.83     1
2 mclapply        2   1.46     1.94
3 mclapply        4   0.841    3.36
4 mclapply        6   0.625    4.52
5 mclapply        8   0.485    5.83
6 mirai_map       2   1.84     1.54
7 mirai_map       4   1.01     2.80
8 mirai_map       6   0.811    3.49
9 mirai_map       8   0.615    4.60
```

The speedup is sublinear in the number of workers for both, and `mclapply()` edges out `mirai_map()` at every count. This is close to an ideal case for forking. The workers inherit the resamples, the grid, and `boot_fit()` for free, where `mirai` has to send each resample to a daemon.

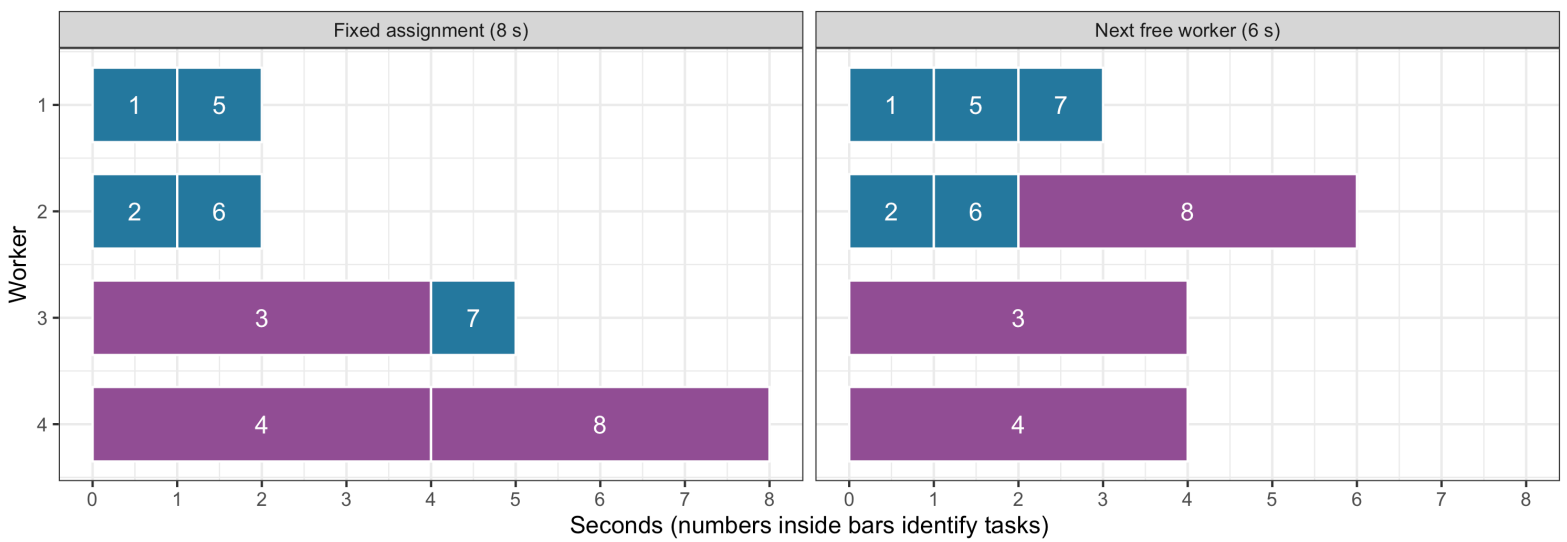
Aside - Scheduling

Eight tasks on four workers, where each task does nothing but sleep for a set number of seconds.

```r
waits = c(1, 1, 4, 4, 1, 1, 1, 4)
```

How long does this take if the tasks are dealt out in order (round robin) or if instead each task goes to the first free worker?

Diagram | Timings



Daemons on other machines

The same code runs with daemons on remote machines. The host listens at a URL and the daemons dial in, launched over SSH or submitted through a cluster scheduler.

```r
daemons(
  url = host_url(),
  remote = ssh_config(c("ssh://hypatia", "ssh://germain"))
)

daemons(
  n = 20, url = host_url(),
  remote = cluster_config(command = "sbatch", options = "#SBATCH --mem=4G")
)
```

See the distributed computing section of the `mirai` docs. `remote_config()` builds a launch command for any other

`mirai` and the tidyverse

in_parallel()

purrr's in_parallel() wraps the function given to any map() variant. When daemons are set the map runs through mirai, otherwise it runs sequentially.

```r
system.time({
  res = tibble(b = boots) |>
    mutate(
      fit = map(b,
        in_parallel(\(b) boot_fit(b, grid), boot_fit = boot_fit, grid = grid)
      )
    )
})
```

```
   user  system elapsed
  0.067   0.271   1.027
```

- in_parallel() creates a self-contained function (a crate) that is serialized and sent to the daemons.
- mirai_map() can be called directly instead, and over a data frame it maps across rows like pmap(). in_parallel() works with the whole map() family, so it fits inside mutate() and keeps purrr's type-stable variants.

Self-contained functions

Use of `in_parallel()` has similar limits to direct use of `mirai()`,

- Package functions must be namespaced, `dplyr::filter()` rather than `filter()`, or the package attached inside the function.
- Objects and helper functions from your session must be passed to `in_parallel()` by name, as in `in_parallel(\(x) f(x), f = f)`. Helpers passed this way must themselves be self-contained.
- `map()`'s `...` is not available for a parallel map, so `map(boots, in_parallel(boot_fit), grid = grid)` does not work. Fix extra arguments inside the lambda instead, as in `\(b) boot_fit(b, grid)`, with `grid` passed to `in_parallel()` by name.

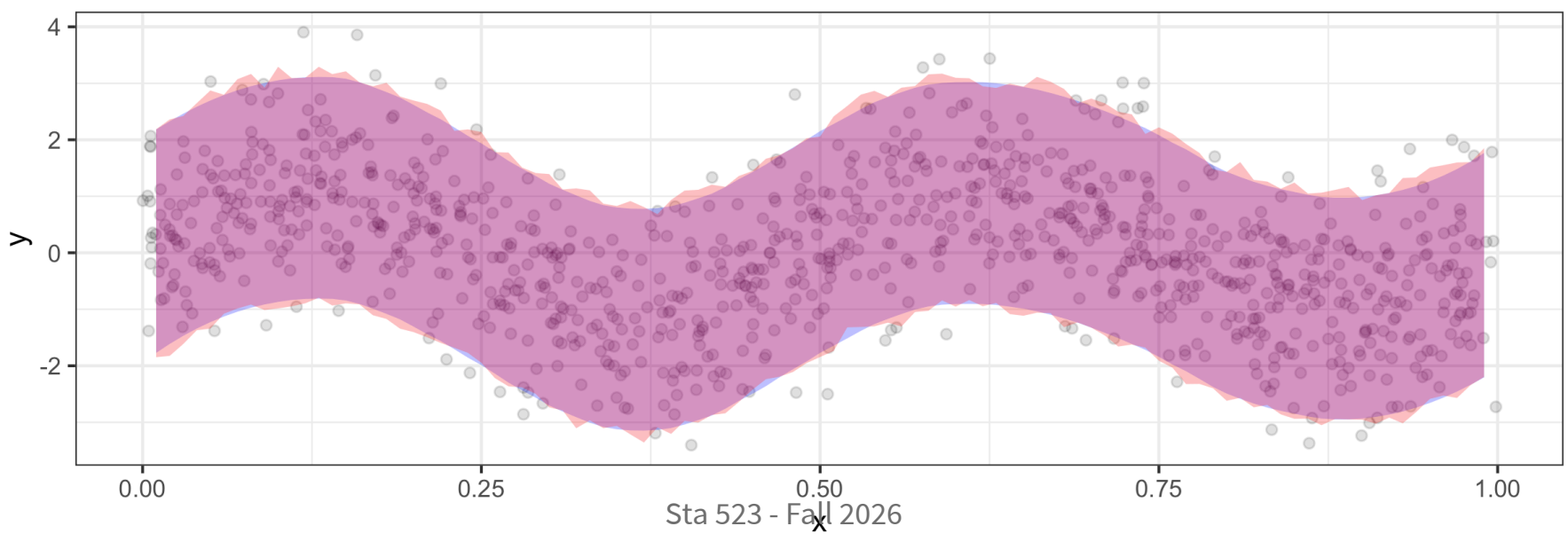
```r
map(boots[1:3], in_parallel(\(b) boot_fit(b, grid)))
```

```
Error in `map()`:
ℹ In index: 1.
Caused by error in `boot_fit()`:
! could not find function "boot_fit"
```

Bootstrap band

Unnest the fits and summarize by `x` to get the bootstrap prediction band, shown in red. The blue band is the prediction interval from the `loess()` fit to the original data,

```r
band = res |>
  select(fit) |>
  tidyr::unnest(fit) |>
  summarize(
    lower = quantile(y_new, 0.025, na.rm = TRUE),
    upper = quantile(y_new, 0.975, na.rm = TRUE),
    .by = x
  )
```



When it does not help

For small calculations, compare a vectorized call, a sequential map, and a parallel map,

```r
daemons(4)
x = runif(1e4)
```

```r
bench::mark(
  vectorized = sqrt(x) + 1,
  sequential = map_dbl(x, \(v) sqrt(v) + 1),
  parallel = map_dbl(x, in_parallel(\(v) sqrt(v) + 1)),
  iterations = 3, filter_gc = FALSE
) |>
  select(expression, median)
```

```
# A tibble: 3 × 2
  expression   median
  <bch:expr> <bch:tm>
1 vectorized  25.26µs
2 sequential   2.05ms
3 parallel      1.03s
```

Python

The Python toolkit

The workflow is the same, measure first and then decide whether workers are worth it. The Python equivalents are mostly in the standard library,

Task	R	Python
Profile	`profvis`	`cProfile` and `pstats`
Time a call	`bench::mark()`	`timeit`
Pools of workers	`mclapply()`, `mirai_map()`	`concurrent.futures`
Persistent workers, session functions	`mirai` daemons, `in_parallel()`	`joblib`

The rest of this section repeats the bootstrap, profiles a replicate, and then compares sequential, thread, and process execution of the same tasks.

`boot_fit()` in Python

The smoother is now `lowess()` from `statsmodels`. Compared to the R version, `boot_fit()` takes a seed rather than a resample and draws the resample itself. A seed is a small thing to send to a worker, where a 1,000 row data frame is not, and the next slides use it to make each task reproducible.

The function lives in its own module, `data/boot_fit.py`, because a pool of processes imports the task function in each worker instead of copying the session the way a fork does,

```py
import numpy as np
from statsmodels.nonparametric.smoothers_lowess import lowess

def boot_fit(seed, d, grid, span = 0.3):
    rng = np.random.default_rng(seed)
    b = d.sample(frac = 1, replace = True, random_state = rng)
    fitted = lowess(b["y"], b["x"], frac = span, it = 0, return_sorted = False)
    pred = lowess(b["y"], b["x"], frac = span, it = 0, xvals = grid)
    return pred + rng.choice(b["y"].to_numpy() - fitted, len(grid))
```

The same data

The same points and prediction grid as in R, and one replicate from the imported function,

```py
import numpy as np
import pandas as pd
from data.boot_fit import boot_fit

rng = np.random.default_rng(3212016)
x = rng.uniform(size = 1000)
d = pd.DataFrame({"x": x, "y": np.sin(4 * np.pi * x) + rng.normal(size = 1000)})
grid = np.linspace(0.01, 0.99, 99)
```

```py
boot_fit(1, d, grid)[:5]
```

```
array([-0.12388181,  0.25259836,  1.69818199, -2.91889708,  2.13303352])
```

Profile before adding workers

`cProfile` is Python's profiler and `pstats` summarizes what it recorded. Profiling 100 replicates and sorting the calls by cumulative time shows where the time goes,

```py
import cProfile, pstats

with cProfile.Profile() as pr:
  fits = [boot_fit(seed, d, grid) for seed in range(100)]

_ = pstats.Stats(pr).strip_dirs().sort_stats("cumulative").print_stats(4)
```

```
         515018 function calls (513618 primitive calls) in 0.434 seconds

   Ordered by: cumulative time
   List reduced from 172 to 4 due to restriction <4>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      100    0.002    0.000    0.434    0.004 boot_fit.py:4(boot_fit)
      200    0.316    0.002    0.414    0.002 smoothers_lowess.py:15(lowess)
    73036    0.024    0.000    0.081    0.000 fromnumeric.py:2342(sum)
    73436    0.027    0.000    0.053    0.000 fromnumeric.py:66(_wrapreduction)
```

Nearly all of the time is inside the two `lowess()` calls. That is library code we cannot rewrite, so the practical option here is more workers.

Random streams

Each worker process has its own copy of the global generator, either copied from the parent or freshly seeded, so `np.random.seed()` in the session does not make the results reproducible. This is the same problem as with forked R workers.

The fix mirrors L'Ecuyer streams. `SeedSequence.spawn()` derives independent streams from one seed, and `boot_fit()` builds its generator from the stream it is given,

```py
reps = 1000
seeds = np.random.SeedSequence(42).spawn(reps)
seeds[0]
```

```
SeedSequence(
    entropy=42,
    spawn_key=(0,),
)
```

A stream belongs to its task, so worker count and scheduling do not change the draws,

```py
np.array_equal(boot_fit(seeds[0], d, grid), boot_fit(seeds[0], d, grid))
```

```
True
```

Sequential baseline

Every strategy below is compared against a sequential run, the Python counterpart of the `lapply()` timing. `partial()` fixes the arguments shared by every task, so `fit(seed)` is a complete task with a single argument, which is what `map()` and the worker pools expect.

`timeit.timeit()` with `number = 1` times one run and discards its result, so the fits are kept from a separate call,

```py
from functools import partial
import timeit

fit = partial(boot_fit, d = d, grid = grid)
sequential = list(map(fit, seeds))

sequential_seconds = timeit.timeit(lambda: list(map(fit, seeds)), number = 1)
sequential_seconds
```

```
3.73728329106234
```

Threads, processes, and the GIL

R's forks and daemons are both processes. Python also offers threads, which share one interpreter, and whether they help depends on the global interpreter lock (GIL),

- With the GIL enabled, only one thread runs Python code at a time. Threads overlap waiting, but do not speed up computation written in Python.
- Compiled code can release the GIL while it runs, but only if it was written to. Measure rather than assume.
- Processes have separate interpreters and GILs. Starting them and transferring inputs and results costs time.

Free threaded Python became officially supported in 3.14 and remains optional. Extension compatibility varies.

Compare execution strategies

`concurrent.futures` provides a thread pool and a process pool with the same interface. The `with` block starts the workers and shuts them down at the end, and `pool.map()` spreads the calls across them and returns the results in input order,

```py
from concurrent.futures import ProcessPoolExecutor, ThreadPoolExecutor

def run(executor):
  with executor(max_workers = 4) as pool:
    return list(pool.map(fit, seeds))

runs = [("sequential", sequential_seconds)]
for executor in [ThreadPoolExecutor, ProcessPoolExecutor]:
  seconds = timeit.timeit(lambda: run(executor), number = 1)
  assert np.array_equal(run(executor), sequential)
  runs.append((executor.__name__, seconds))
```

The timings include pool startup and shutdown. `timeit` discards the return value, so each executor runs a second time to check every fitted value against the sequential run. Only the execution changes, not the tasks or their results.

Does parallelization still help?

Speedup is the sequential time divided by each method's time,

```py
comparison = pd.DataFrame(runs, columns = ["method", "seconds"])
comparison["speedup"] = sequential_seconds / comparison["seconds"]
comparison
```

	method	seconds	speedup
0	sequential	3.737283	1.000000
1	ThreadPoolExecutor	3.874532	0.964577
2	ProcessPoolExecutor	1.606327	2.326602

Threads take as long as the sequential run. `lowess()` is Python code, so only one thread holds the GIL at a time.

Four processes finish in well under half the time, and that includes starting four interpreters that each import `statsmodels`. The next slides measure that startup cost.

`joblib` and repeated calls

`joblib` is the package `scikit-learn` uses for parallel work, and it lifts two limits of the process pool. Its default backend can send functions defined in the session, like `in_parallel()`'s crate, and it keeps the workers alive between calls, like daemons.

`delayed(fit)(seed)` records a call without running it and `Parallel` runs the batch. Timing two calls in a row shows the effect of reusing the workers,

```py
from joblib import Parallel, delayed

def run_joblib():
  return Parallel(n_jobs = 4)(delayed(fit)(seed) for seed in seeds)

joblib_seconds = [timeit.timeit(run_joblib, number = 1) for _ in range(2)]
assert np.array_equal(run_joblib(), sequential)
```

Account for startup

```py
pd.DataFrame({"call": ["first", "repeated"], "seconds": joblib_seconds})
```

```
       call   seconds
0     first  1.434059
1  repeated  0.973770
```

The first call starts four workers that each import the task's modules. The repeated call reuses them, so it is faster again. Both calls run the same tasks and reproduce the sequential results.

For a one-off script, include startup in the comparison. For repeated work in a session, also measure a call with workers already running.

Summary

Takeaways

- Profile before optimizing. `profvis` attributes time to lines of your code, and the slow line is rarely the one you expect. A better algorithm usually beats more cores.
- Check that alternatives do the same work before comparing speed. `bench::mark()` checks results, and `bench::press()` varies input size and group count.
- `mclapply()` forks the session for parallel `lapply()` on Unix. Its default fixed assignment can leave cores idle; `mc.preschedule = FALSE` schedules tasks as slots free up.
- Time asynchronous work from submission through collection. `mirai` returns errors as values, so successful results can be kept while failed tasks are repaired and retried.
- `purrr::in_parallel()` and `mirai_map()` put parallel work inside pipelines. Assign reproducible random streams to tasks so worker count does not change the results.
- In Python, measure threads and processes on the real workload, accounting for pool startup and worker reuse. Threads only help when the hot code releases the GIL.

Learning more

- profvis and the Measuring performance chapter of Advanced R
- bench
- mirai, with articles on daemons and parallel maps
- Parallelization in purrr
- concurrent.futures and joblib
- Parallel random streams in NumPy

Reference

Quick Python timings

In Jupyter and IPython, `%timeit` chooses the number of loops and repeats itself,

```py
%timeit boot_fit(1, d, grid)
```

With `timeit.repeat()`, take the minimum as an estimate of execution time with the least interference from other work on the machine.

Timeouts and cancellation

A timeout returns an error value. With a dispatcher, it also cancels the task.

```r
daemons(4)
m = mirai(Sys.sleep(10), .timeout = 500)
m[]
is_error_value(m$data)
```

`.timeout` is in milliseconds. `stop_mirai()` cancels a task explicitly.

Task size and nested parallelism

`purrr` suggests roughly 100 µs to 1 ms of work per task as a starting point for considering parallelization. Measure on your own workload.

Some libraries already use multiple threads, including `data.table`, `polars`, `duckdb`, and optimized BLAS libraries.

Running multithreaded code in several worker processes can create more runnable threads than available cores. Limit internal threads or reduce the worker count, then measure again.

Other packages

- `future` and `furrr` are an older framework with the same idea, a plan is set once and `future_map()` runs on it. `mirai` is now one of its backends.
- `foreach` with `doParallel` or `doMC` is a loop based interface that is still common in older code and packages.
- `parallel::parLapply()` and `makeCluster()` create socket workers that work on Windows but need explicit `clusterExport()` of every object.


mizu
ミズ

`mizu`

`mizu` is a new package from the author of `mirai` for parallel R on a single machine,

- workers are background R processes, like daemons, but tasks and results move through shared memory rather than sockets,
- a task round trip takes about a microsecond, against roughly 100 for `mirai`, so work can be split much more finely,
- a parallel map places the function and data in shared memory once, and the workers divide the elements among themselves,
- `.seed` gives each element of a map its own random number stream, so results do not depend on the number of workers,
- shared memory only reaches the same machine, so `mirai` is still the tool for remote machines and clusters.

`mizu_pool()` and `mizu_map()`

`mizu_pool()` starts the workers, fresh R processes like daemons. `mizu_map()` returns a handle immediately and `mizu_collect()` waits for the results, a list as with `lapply()`. Here ten tasks each draw a million normals and return their standard deviation,

```r
sizes = rep(1e6, 10)
```

```r
p = mizu_pool(n_workers = 4)
(m = mizu_map(p, sizes, \(n) sd(rnorm(n))))
```

```
<mizu_map: 10 elements, inline blob, submitted>
```

```r
unlist(mizu_collect(m))
```

```
 [1] 0.9993297 1.0009510 0.9994861 1.0000613 1.0003001 0.9998641 1.0007215
 [8] 1.0003905 0.9985753 1.0007674
```

Adding `.template = numeric(1)` returns a double vector directly, as `vapply()` or `map_dbl()` would.

Smaller pieces

To compare the three, fix the total work at ten million draws and cut it into more and smaller pieces. Each method gets four workers and the same function, and each piece returns a single number.

```r
daemons(4)
f = \(n) sd(rnorm(n))
elapsed = \(expr) system.time(expr)[["elapsed"]]
```

```r
time_pieces = function(pieces) {
  sizes = rep(1e7 / pieces, pieces)
  tibble(
    pieces,
    lapply = elapsed(lapply(sizes, f)),
    mclapply = elapsed(mclapply(sizes, f, mc.cores = 4)),
    mirai_map = elapsed(mirai_map(sizes, f)[]),
    mizu_map = elapsed(mizu_collect(mizu_map(p, sizes, f)))
  )
}
```

What splitting costs

```r
(times = map(10^(1:5), time_pieces) |> list_rbind())
```

```
# A tibble: 5 × 5
  pieces lapply mclapply mirai_map mizu_map
   <dbl>  <dbl>    <dbl>     <dbl>    <dbl>
1     10  0.170   0.0920    0.0640   0.0510
2    100  0.168   0.0750    0.0580   0.0480
3   1000  0.272   0.0890    0.122    0.0450
4  10000  0.200   0.0940    1.08     0.0550
5 100000  0.475   0.221    10.6      0.141
```

- `mclapply()` deals the pieces out in four batches, one per forked worker, so more pieces cost it almost nothing.
- `mirai_map()` sends every piece as its own task, at roughly 100 µs each. That is 10 seconds for 100,000 pieces.
- `mizu_map()` workers claim pieces in batches from shared memory, so its time follows the work even at 100,000 pieces.
- By the last row the pieces are so small that 100,000 calls to `f()` cost more than the draws, which slows every method.

Python workers

`pymizu` is the Python side of `mizu`. A pool given `mizu_py_launcher()` has Python processes as its workers, here from the course environment where `pymizu` is installed.

```R
py = mizu_pool(
  n_workers = 4,
  launcher = mizu_py_launcher(python = ".venv/bin/python")
)
```

`mizu_call()` names a Python function and its arguments, unnamed ones positional and named ones as keywords. `mtcars$mpg` arrives as a NumPy array and the result returns as an R vector.

```R
t = mizu_submit_call(py, mizu_call("numpy.quantile", mtcars$mpg, q = c(0.25, 0.5, 0.75)))
mizu_collect(t)
```

```
[1] 15.425 19.200 22.800
```

```R
quantile(mtcars$mpg, c(0.25, 0.5, 0.75))
```

```
   25%    50%    75%
15.425 19.200 22.800
```

What crosses

Values convert to their natural counterpart. Vectors and matrices arrive as NumPy arrays, length 1 vectors as scalars, character vectors as lists of `str`, and named lists as `dict`s.

Large values are written to shared memory once and both sides read them in place, so nothing is serialized or copied again. Summing 80 MB of doubles in a Python worker takes about as long as `sum()` in the session,

```r
x = runif(1e7)
```

```r
system.time(mizu_collect(mizu_submit_call(py, mizu_call("numpy.sum", x))))
```

```
   user  system elapsed
  0.006   0.006   0.018
```

```r
system.time(sum(x))
```

```
   user  system elapsed
  0.013   0.000   0.013
```

Data frames

A `.source` string runs a block of Python with the named arguments bound as variables, and its last expression is the result. A data frame arrives as a `pymizu.Frame`, which `polars`, `pyarrow`, and `pandas` can read, and a frame returned comes back as a data frame,

```r
by_cyl = mizu_call(.source = r"(
import polars as pl
(pl.DataFrame(d)
   .group_by("cyl")
   .agg(pl.col("mpg").mean())
   .sort("cyl"))
)", d = mtcars)
mizu_collect(mizu_submit_call(py, by_cyl))
```

```
  cyl      mpg
1   4 26.66364
2   6 19.74286
3   8 15.10000
```

The same task in Python

With a `mizu_call()` as the function, `mizu_map()` runs Python over an R vector. Each element is bound to `x`, and `.template` collects the results into a double vector. These are the ten pieces of a million draws from before, now drawn by NumPy,

```R
sd_py = mizu_call(.source = r"(
import numpy as np
np.random.default_rng().standard_normal(int(x)).std(ddof = 1)
)")
```

```R
m = mizu_map(py, sizes, sd_py, .template = numeric(1))
mizu_collect(m)
```

```
[1] 1.0004404 1.0004014 0.9998788 1.0009277 1.0009393 1.0009254 1.0003031
[8] 1.0010922 1.0002947 0.9998401
```